

Metodo delle Correnti di Maglia

Programmazione e Calcolo Scientifico 2025–2026

Catellani Diego Colucci Angelamaria Colucci Giorgia

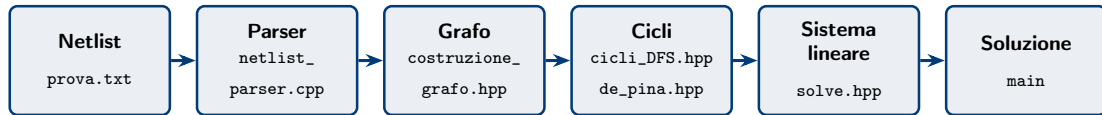
Politecnico di Torino

A.A. 2025–2026

Panoramica del progetto

Il programma risolve circuiti elettrici tramite il **metodo delle correnti di maglia**:

- **Parsing** robusto della netlist: lettura e validazione di resistori (R) e generatori ideali di tensione (V).
- **Costruzione del grafo** topologico del circuito.
- **Individuazione delle maglie** – due algoritmi alternativi:
 - DFS + coalbero – cicli fondamentali;
 - De Pina – base di cicli minimi;
- **Assemblaggio e risoluzione** del sistema lineare $(B^T R B) i = v$ tramite Eigen.
- **Stampa** di tensione e corrente su ogni resistore.
- **Test unitari** su parser, costruzione del grafo, algoritmi dei cicli e solver.



Parsing della netlist

Legge una netlist e restituisce un `Output{ok, vector<Componente>}`, dove ogni `Componente` contiene cinque campi:

TIPO	NOME	VALORE	NOD01	NOD02
Resistore	R1	20	1	2

Classificazione delle anomalie

- **Tollerate**: nessun messaggio
(whitespace, prefisso minuscolo, nodi tipo 1.0)
- **Warning**: avviso su cerr e prosegue con scelta di default
(campi extra, $R < 0$, nomi duplicati, componenti in parallelo)
- **Errori**: interrompe subito con `ok=false`
($R = 0$, prefisso $\notin \{R, V\}$, nodi coincidenti, indice ≤ 0 , nodi non interi)

Scelte di design

- Niente eccezioni: il flag `ok` di `Output` segnala il fallimento al chiamante.
- `std::getline` + `std::stringstream` per la robustezza ai whitespace senza parsing manuale.

Costruzione del grafo

Scelta di design

Una sola mappa $\text{edge}\langle\text{int}\rangle \rightarrow \text{edge_data}$ raccoglie tutto in modo atomico.

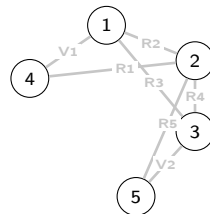
```
struct edge_data {  
    TipoComponente tipologia; // Resistore |  
    Generatore  
    double        valori;    // Ohm o Volt  
    bool          verso_positivo; //  $n_1 < n_2$   
    std::string    nome;  
};
```

Difficoltà: ogni arco porta 4 informazioni; mappe separate rischierebbero disallineamenti e bug difficili da individuare.

Precisione: `verso_positivo` è calcolato per *tutti* i componenti, ma usato in solve solo per i generatori.

Contenuto della mappa (prova.txt)

nome	valore	verso pos.	tipo
V1	30 V	true (1<4)	Generatore
R2	10 Ω	true (1<2)	Resistore
V2	40 V	true (3<5)	Generatore
R1	4 Ω	false (4>2)	Resistore



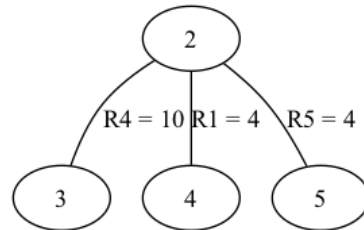
$$n = 5 \text{ nodi}, m = 7 \text{ archi}, k = m - n + 1 = 3 \text{ maglie.}$$

Cicli fondamentali: DFS + coalbero

- **Spanning tree** T – `recursive_dfs()` visita G ricorsivamente da una radice e costruisce l'albero di copertura T .
- **Coalbero** $C = G \setminus T \rightarrow k$ **cicli** – `operator-(G,T)` filtra gli archi non in T ; i $k = m - n + 1$ archi di coalbero determinano k cicli fondamentali.
- **Ciclo** $j = \text{path in } T + \text{arco di coalbero}$ – per ogni arco $(a, b) \in C$: `find_path(T,a,b)` + chiusura implicita $b \rightarrow a$.

Il cammino in T tra a e b è *unico* (T è un albero): si usa `dfs_path()` (DFS ricorsivo con backtracking).

Coalbero (output con DFS)



Perché esiste anche De Pina?

Il DFS *non* garantisce cicli minimi: dipendono da radice e ordine di visita. De Pina garantisce la **base di cicli di lunghezza minima**.

Costruisce una **base di cicli minimi** (MCB) in $\mathbb{Z}/2\mathbb{Z}$:

- Albero di ricoprimento e famiglia di vettori di supporto S_i (uno per arco di coalbero, $S_i[e_i] = 1$).
- Per ogni i : si seleziona il ciclo minimo C_i^* con $\langle C_i^*, S_i \rangle_{\mathbb{Z}_2} = 1$.
- Aggiornamento dei vettori successivi: $S_j \leftarrow S_j \oplus S_i$ se $\langle S_j, C_i^* \rangle_{\mathbb{Z}_2} = 1$ (prodotto scalare e differenza binaria $\Rightarrow$ indipendenza lineare).
- Ricostruzione della base di cicli minimi.

Difficoltà incontrate e scelte implementative

- Tipo di ritorno a vettori di *nodi* (non booleani di archi): accessi successivi più comodi.
- $|V| + 1$ come caso peggiore per gestire la maglia singola.
- Gestione degli “infiniti” con sentinelle.
- Ricostruzione dei cicli di nodi dai booleani di archi: gestione degli zeri iniziali.

$$(B^{\top}RB)i = v \quad m \text{ componenti, } k \text{ maglie, } B \in \mathbb{R}^{m \times k}, R \in \mathbb{R}^{m \times m} \text{ diag.}, v \in \mathbb{R}^k.$$

Matrice B – incidenza ciclo–arco

$$B_{ij} = \begin{cases} +1 & \text{verso canonico from} \rightarrow \text{to} \\ -1 & \text{verso opposto to} \rightarrow \text{from} \\ 0 & \text{arco } i \text{ non nel ciclo } j \end{cases}$$

Vettore v – contributo generatori

$v[j] += \text{dir} \times \text{sgn} \times V$, con $\text{dir} = B[i][j]$, $\text{sgn} = (\text{verso_positivo?} - 1 : +1)$.
Entra da $-$, esce da $+$ $\Rightarrow +V$; entra da $+$, esce da $-$ $\Rightarrow -V$.

Solve: esempio dei segni e scelta del solver

Esempio – V1 (30 V) in prova.txt

V1 30 1 4 $\rightarrow n_1=1, n_2=4$; verso_positivo = $(1 < 4) = \text{true} \Rightarrow$ terminale + al nodo 1.

Verso 1 $\rightarrow$ 4: dir = +1, sgn = -1, $v[j] += (+1)(-1)(30) = -30$

Verso 4 $\rightarrow$ 1: dir = -1, sgn = -1, $v[j] += (-1)(-1)(30) = +30$

Perché il gradiente coniugato?

$A = B^T R B$ è simmetrica e (con almeno un resistore per maglia) definita positiva $\Rightarrow$ il gradiente coniugato converge senza fattorizzazione esplicita, in al più k iterazioni.

Costi computazionali – riepilogo

Funzione	File	Costo	Note
parse_netlist	netlist_parser.cpp	$O(m^2 + S)$	check duplicati/paralleli quadratici; S = dim. input
costruisci_grafo	grafo_construction.cpp	$O(m^2 \log m)$	add_edge riordina il vettore archi a ogni inserimento
recursive_dfs	recursive_DFS.hpp	$O(n^2 \log n)$	costruzione albero T ($n - 1$ add_edge ordinanti)
cicli_DFS	cicli_DFS.hpp	$O(n^2 \log n)$	albero T + k cammini dfs_path ($O(k n \log n)$)
De_Pina	de_pina.hpp	$O(k(n^2 m + m^2 \log(m)))$	k iter. $\times$ n Dijkstra su G' ($2n$ nodi, $2m$ archi), in circuiti sparsi $O(kn^3)$
circuit (solver)	solve.hpp	$O(k m^2 + k^3)$	$A = B^T R B$ (R densa) + CG ($\leq k$ iter. da $O(k^2)$)
graph_visit	graph_visit.hpp	$O(n^2 \log n)$	usata in solve (connessione) e main (export)
print_graphviz	stampa_grafi.hpp	$O(m)$	una sola passata sugli archi

$n = |V|$ nodi, $m = |E|$ componenti, $k = m - n + 1$ maglie. I fattori m^2/n^2 derivano dalla scelta del vettore ordinato

Test unitari (1/2)

test_parser.cpp – 20 test

Categoria	Test
Parsing pulito	7
Warning (prosegue)	5
Errori fatali	8
Totale	20

Quando uno fallisce si sa esattamente cosa è rotto.

test_grafo.cpp

Costruzione da prova.txt (5 nodi, 7 componenti):
legge con `parse_netlist`, costruisce con
`costruisci_grafo`, verifica nodi e archi.

test_solve.cpp

Verifica che la soluzione del circuito (correnti/tensioni)
coincida con i valori attesi, usando la base di cicli da
DFS oppure da De Pina.

Test unitari (2/2)

test_cicliDFS.cpp

Cicli fondamentali via DFS + coalbero, sugli *stessi* grafi di test_de_pina.cpp per il confronto diretto:

- F_albero: 4 nodi, 3 archi – un albero, nessun ciclo;
- G_ciclico: F_albero + 2 archi – 2 cicli fondamentali ($= \text{archi} - \text{nodi} + 1$).

test_de_pina.cpp

- Base vuota per un albero (senza cicli).
- Ciclo minimo per un grafo che è solo un ciclo.
- Cicli minimi per un grafo con cicli semplice.

Difficoltà: confronto robusto tra soluzione e output di De Pina – normalizzazione a insieme di archi tramite funzioni helper (a prescindere da ordine e orientamento).

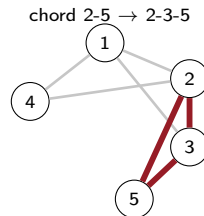
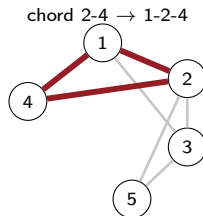
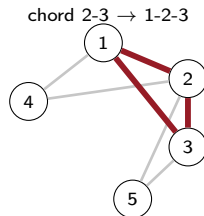
Confronto: base di cicli fondamentali – DFS vs De Pina

Stesso grafo (prova.txt, $|V| = 5$, $|E| = 7$): in questo caso DFS produce già la base minima, coincidente con quella di De Pina.

Cicli da DFS

su spanning tree

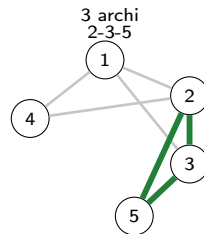
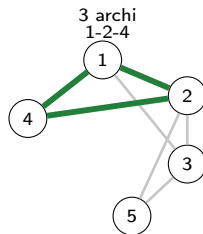
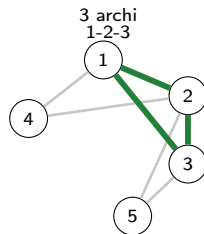
totale: 9 archi



Cicli da De Pina

base di peso minimo

totale: 9 archi



3 archi

3 archi

3 archi

Conclusioni: punti di forza del progetto

Correttezza indipendente

Due metodi per matrici di incidenza con risultati identici. Coincidenza con gli esempi PDF e test suite verde.

Verifica di De Pina

Normalizzazione a insiemi di archi (`normalize_basis`): confronto robusto a prescindere da ordine e orientamento.

Ingegnerizzazione

C++20, CMake, CTest. Struttura modulare (`include/`, `src/`, `test/`) e build pulita senza warning.

Qualità del codice

Separazione I/O, dati coesi in `edge_data`, solidità via `const-correctness` e template generici. Zero rischi ODR.

Robustezza e segni

Gestione di casi degeneri e residui relativi. Cura meticolosa per segni e orientamenti, sfida centrale del progetto.

Funzionalità extra

Esportazione Graphviz (`.dot`) del circuito e del coalbero per la visualizzazione grafica immediata.

Grazie per l'attenzione!

Metodo delle Correnti di Maglia

Programmazione e Calcolo Scientifico 2025–2026